Sports Content Generator Report

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Jaspreet Kour Rathod	EN22CS301456
2	Kartik Dubey	EN22CS301486
3	Jyoti Singh Yadav	EN22CS301473
4	Kapil Daunde	EN22CS301482
5	Madhur Bhawsar	EN22CS301560
6	Kunal Patidar	EN22CS301537

Group Name:10D5

Project Number: GAI-10

Industry Mentor Name: Prof Aashruti Shah

University Mentor Name: Prof. Vineeta Rathore

Table of Contents

1. Introduction
2. System Overview
3. Multi-Agent System Architecture
4. The Content Generator (Prompt Engineering & Execution)
5. The Content Vault (Vector Database & RAG)
6. Interfaces & System Requirements
7. Non-Functional Requirements
8. References

1. Introduction

The project titled **"Sports Content Generator"** (SportsCraft AI) is a Generative AI-powered application developed to assist sports professionals in generating structured, publication-ready sports content automatically.

Unlike traditional chatbots, this platform utilizes a highly sophisticated **Multi-Agent AI workflow (CrewAI)** and **Retrieval-Augmented Generation (RAG)**. The system autonomously researches live match data, validates statistical accuracy, and applies strict Prompt Engineering templates to generate content like Match Recaps and Player Profiles. Furthermore, it utilizes a Vector Database to permanently store generated articles for semantic retrieval.

2. System Overview

The Sports Content Generator focuses heavily on two core Generative AI components:

1. **The Content Generator (The Brain):** A four-agent autonomous pipeline powered by the Groq API (LLaMA 3 models). It processes user inputs, scrapes live sports APIs and web scorecards, cross-checks the data, and formats it using specialized journalism templates.

2. **The Content Vault (The Memory):** A ChromaDB vector storage system that converts generated text into high-dimensional embeddings, allowing users to search their past articles using natural language.

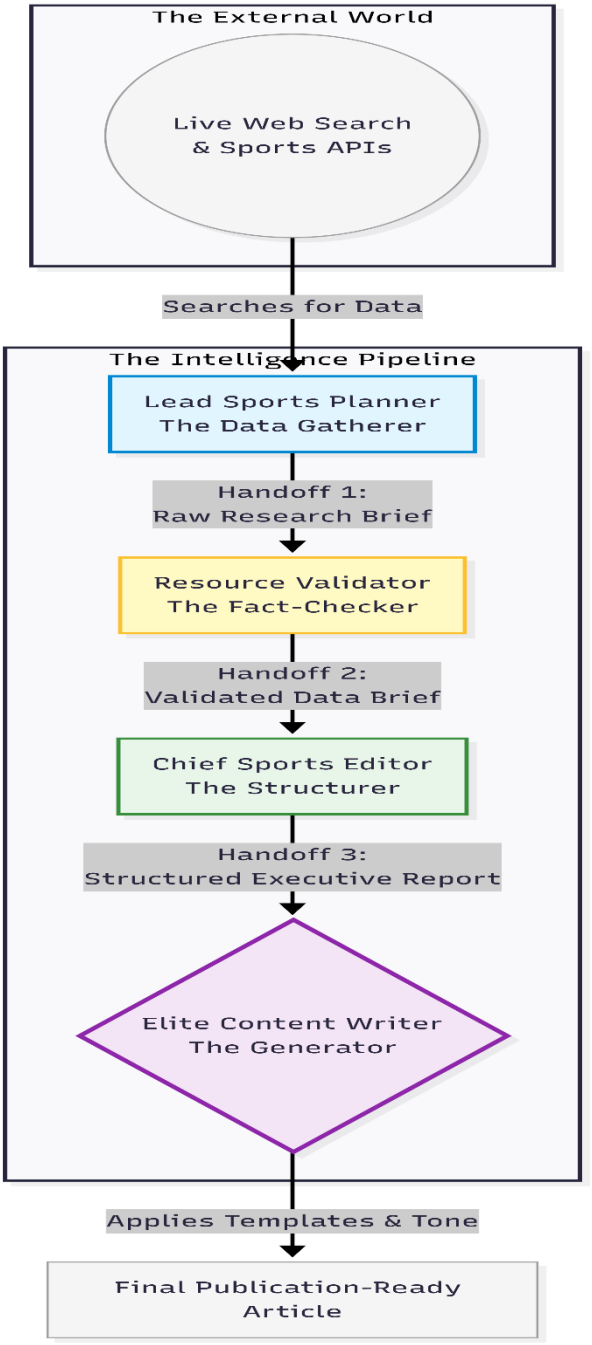
3. Multi-Agent System Architecture

The application departs from a standard linear script and uses a **Decoupled Agentic Architecture**.

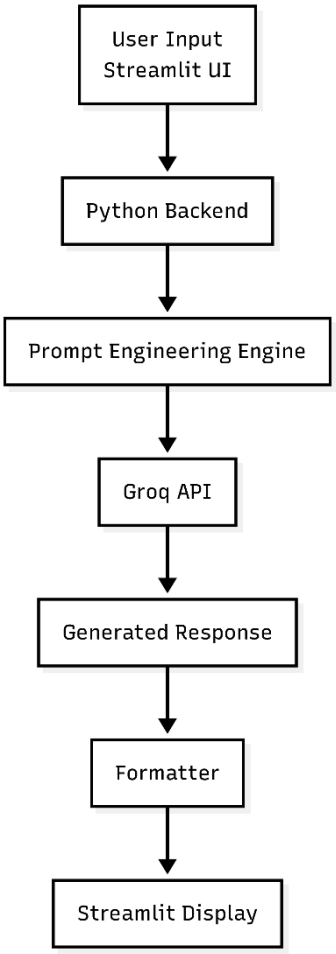
3.1. Agent Roles & Process Flow:

- **Lead Sports Planner (Researcher):** Connects to the Serper Web Search API and Cricket Data APIs to fetch live scores and context.
- **Resource Validator (Analyst):** Acts as the "Hallucination Gatekeeper." It verifies the math and data fetched by the Planner and assigns a Confidence Score.
- **Chief Sports Editor (Structurer):** Takes the validated data and builds Markdown tables and technical summaries.

- **Elite Content Writer (Generator):** Takes the final structured data and applies specific tone and sports terminology . This agent is deliberately restricted from searching the web to ensure it only writes about fact-checked data.


The External World
Live Web Search
& Sports APIs
Searches for Data
The Intelligence Pipeline
Lead Sports Planner
The Data Gatherer
Handoff 1:
Raw Research Brief
Resource Validator
The Fact-Checker
Handoff 2:
Validated Data Brief
Chief Sports Editor
The Structurer
Handoff 3:
Structured Executive Report
Elite Content Writer
The Generator
Applies Templates & Tone
Final Publication-Ready
Article

3.2. Information Flow:



4. The Content Generator (Prompt Engineering)

The core of the GenAI application relies on strict **Prompt Engineering**. Instead of allowing the LLM to write freely (which leads to hallucinations and inconsistent tones), the system uses predefined "GAI-10 Templates."

- **Dynamic Variable Injection:** The system dynamically injects variables such as {topic}, {sport}, and {tone} into system prompts.
- **Zero-Hallucination Guardrails:** The LLM is explicitly instructed to output "Data Unavailable" rather than inventing statistics if the Planner agent fails to find a specific metric (e.g., a player's exact strike rate).
- **Tone Control:** The generator adjusts its vocabulary based on user selection (e.g., using intense, engaging language for a "Dramatic" recap versus analytical language for a "Journalistic" report).

5. The Content Vault (Data Design & RAG)

Because this is a Generative AI project, standard relational databases (like MySQL or SQLite) are insufficient for semantic understanding.

5.1. Vector Database Integration (ChromaDB) The system utilizes **ChromaDB** to manage persistent storage (./sports_vault_db).

- When the Content Generator finishes an article, the text is passed through an Embedding Model.
- The text is converted into Vectors and mapped alongside metadata (e.g., Date, Sport type).
- **Semantic Search:** When a user searches the Content Vault, the system matches the *meaning* of their query against the vectors, rather than relying on exact keyword matches.

6. Interfaces & Caching

6.1. User Interface (Streamlit) The user interface is a modern, glassmorphism-styled dashboard. It is divided into:

- **The Generator Tab:** Where users input Topics, Sports, and select Tones.
- **The Vault Tab:** A dedicated semantic search engine to browse previously generated reports.

6.2. State and Caching

- **Streamlit Session State:** Maintains UI stability between refreshes and stores temporary chat history.
- **Caching:** Implementation of decorators like @st.cache_data prevents redundant calls to the Groq API for identical prompts, saving tokens and reducing latency.

7. Non-Functional Requirements

7.1. Security Aspects

- **Secret Management:** API keys (Groq API, Serper API) are securely stored in .env files and never hardcoded into the repository.
- **Data Privacy:** The application does not store sensitive personal user data.

7.2. Performance Aspects

- **Inference Speed:** By utilizing the Groq API (LPU architecture) instead of standard LLM APIs, the multi-agent reasoning process is executed with ultra-low latency.
- **Token Optimization:** Web search payloads (Serper) are strictly limited to prevent overloading the LLM's context window.

8. References

1. Streamlit Inc. (2024). Streamlit Documentation. Available at: https://docs.streamlit.io
2. CrewAI Framework Documentation. Multi-Agent Orchestration.
3. Groq API Documentation. High-Speed LPU Inference.
4. ChromaDB Documentation. AI-native open-source vector database.